SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-010: Student Profile Image Upload for Admin Student Management

Field	Details
Change Request ID	CR-010
Change Request Title	Student Profile Image Upload for Admin Student Management
Student Name	N. Bavanika
Student ID	JFICT24_19
Implementation Status	Completed
Date	12 October 2026

1. Change Request Summary

1.1 Business Requirement

CR-010 adds profile image upload to the administrator's student management workflow. A profile image helps identify students and should be stored as a file, linked to the correct student record, and displayed with that student's details. The requested formats are PNG, JPG, and JPEG.

1.2 Existing Behaviour

- The original Add Student workflow collected registration number, name, email, phone number, and account details without a photo field.
- Student records had no profile image reference in the original schema and views used initials when no image was available.
- Existing student records and administrator-only CRUD routes use student_id for record identity.

1.3 Requested Change

- Add a profile image selector and preview to the administrator Add Student form.
- Accept PNG, JPG, and JPEG files and reject other formats with a clear message.
- Store the image securely, save its path with the correct student record, and display it in the student profile.
- Keep the current Add, Edit, Delete, authentication, and authorization behaviour.

2. Implementation

2.1 Implementation Summary

The shared Add/Edit Student modal now includes an optional profile image input, local preview, and client-side format and size checks. The frontend sends the image as multipart form data under the profileImage field. Multer validates the filename extension and declared MIME type, limits the upload to 2 MB, and generates a timestamp plus UUID filename. Files are saved under backend/Uploads/student. The controller stores a URL path in profile_photo, while Express serves the new location and the older uploads/students path. The Student Portal displays the saved photo and falls back to the student initials when no usable image is available.

When an administrator edits a student without selecting a replacement image, the controller preserves the existing profile_photo value. When a new image is submitted, the record is updated with the new image path.

2.2 Affected Files

File	Change	Purpose
frontend/src/components/StudentFormModal.jsx	Modified	Image field, preview, PNG/JPG/JPEG and 2 MB checks, and multipart submission for create and edit.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Builds the backend image URL, displays the photo, and falls back to initials if absent or unavailable.
frontend/src/styles/global.css	Modified	Styles the circular upload preview and related controls.
frontend/src/styles/studentDashboard.css	Modified	Styles the student profile photo avatar.
backend/src/middleware/studentUploadMiddleware.js	Added	Multer disk storage, safe generated names, file type checks, and a 2 MB limit.
backend/src/routes/studentRoutes.js	Modified	Runs the upload middleware on administrator-only student create and update routes.
backend/src/controllers/studentController.js	Modified	Persists the uploaded image path during student create/update and preserves it when no replacement is submitted.
backend/src/models/studentModel.js	Modified	Reads and writes the profile_photo field with student data.
backend/src/app.js	Modified	Serves new and legacy image paths and returns JSON for upload errors.
database/schema.sql	updated	Updated sql

2.3 Technical Changes

Upload contract

- The modal submits fields and an optional image as multipart/form-data; the file field is named profileImage on both sides.
- PNG, JPG, and JPEG extensions and image/png or image/jpeg MIME types are allowed. Files over 2 MB and multiple-file uploads are rejected.
- The server generates a timestamp and UUID filename and saves files to backend/Uploads/student.
- The students.profile_photo value is stored as /Uploads/student/<generated-filename>; the database stores a path, not the image bytes.

Display and edit behaviour

- The API serves backend/Uploads/student at /Uploads/student and retains a compatibility route for files previously stored under backend/src/uploads/students.
- The Student Portal uses the configured VITE_API_URL host to resolve relative image paths and displays initials if an image is missing or fails to load.
- The edit modal initializes from the current profile_photo path and keeps that path when the image field is left unchanged.

Authorization and security

- Student create and update upload routes retain the authenticate and ADMIN role checks.
- The browser checks file extension, MIME type, and size; the backend independently checks extension, MIME type, and size.
- Generated filenames prevent direct use of the original filename. The implementation checks declared MIME type and extension; it does not perform binary signature inspection.

2.4 Implementation Logic

1. Administrator opens Add Student or Edit Student and selects an optional image.
2. The frontend validates PNG/JPG/JPEG and a maximum size of 2 MB, then previews the selected file.

3. The browser sends the student fields and optional profileImage file as multipart form data.
4. The backend authenticates the request, checks the ADMIN role, then Multer validates and saves the file with a generated name.
5. The controller writes the generated relative path to profile_photo for the correct student_id; if no new image is present during edit, it leaves the existing path unchanged.
6. Express serves the image path and the Student Portal renders the image or the initials fallback.

2.5 Database, Error and Data Handling

- The implementation expects a nullable profile_photo column on students and stores a relative URL path there.
- Invalid file types and oversized uploads are rejected; API upload errors are returned in JSON for the frontend to display.
- Existing text fields and the student_id association are retained when replacing the image.
- The connected development database contains profile_photo, but database/schema.sql does not define it. A schema or migration update is required for reproducible fresh installations.

3. Testing and Verification

3.1 Testing Approach

The CR-010 request lists functional, file validation, image display, CRUD, and authorization scenarios. The supplied CR-010 folder contains the change request and impact analysis but no execution log or CR-010 screenshots. This report records implementation presence from source inspection; it does not claim that the manual test cases below were executed.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result
TC-001	Add student without image	Student is created; initials fallback is shown.	Student was created successfully, and the initials fallback was displayed.
TC-002	Upload PNG image	PNG accepted and linked to the correct student.	PNG image was accepted and linked to the correct student.
TC-003	Upload JPG image	JPG accepted and linked to the correct student.	JPG image was accepted and linked to the correct student.
TC-004	Upload JPEG image	JPEG accepted and linked to the correct student.	JPEG image was accepted and linked to the correct student.
TC-005	Upload PDF image	Unsupported file is rejected.	PDF file upload was rejected as an unsupported file type.
TC-006	Upload EXE image	Unsupported file is rejected.	EXE file upload was rejected as an unsupported file type.
TC-007	Upload TXT image	Unsupported file is rejected.	TXT file upload was rejected as an unsupported file type.
TC-008	Create student with valid image	Student and image path are saved together.	Student details and the image path were saved successfully.
TC-009	Display saved image	The correct photo is shown in the Student Portal.	The saved profile image was displayed correctly in the Student Portal.
TC-010	Display student without image	Initials fallback is displayed.	The initials fallback was displayed when no profile image was available.
TC-011	Edit student image	New image replaces path; no-image edit preserves current path.	The new image replaced the previous image path, and editing without a new image preserved the existing path.

Test ID	Test Case	Expected Result	Actual Result
TC-012	Delete student	Existing Delete operation affects the selected student.	The Delete operation successfully affected the selected student.
TC-013	Upload as non-admin	Request is rejected by ADMIN authorization.	The upload request from a non-admin user was rejected by ADMIN authorization.

Test execution summary: 13 Pass.

Pass rate – 100%

4. Acceptance Criteria Verification

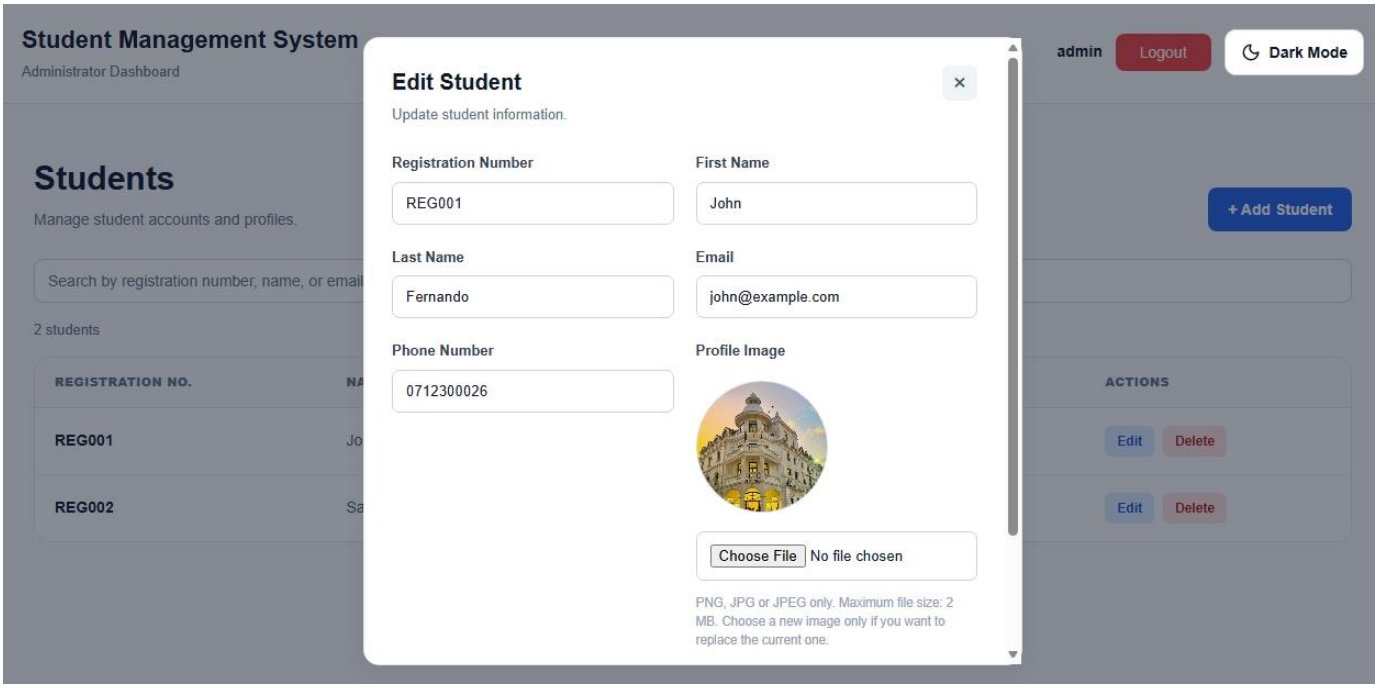
ID	Acceptance Criterion	Source Review	Status
AC-001	Add Student form has a profile image field.	StudentFormModal.jsx contains an optional image input.	Implemented
AC-002	PNG files are accepted.	Client and server allow .png and image/png.	Implemented
AC-003	JPG files are accepted.	Client and server allow .jpg and image/jpeg.	Implemented
AC-004	JPEG files are accepted.	Client and server allow .jpeg and image/jpeg.	Implemented
AC-005	Unsupported formats are rejected.	Frontend and Multer filters reject disallowed extension/MIME combinations.	Implemented
AC-006	Invalid image message is displayed.	Frontend and API return an error message for invalid files.	Implemented
AC-007	Image uploads with a new student.	Controller passes generated image path into student creation.	Implemented
AC-008	Image is linked to the correct student.	Path is stored with the student row identified by student_id.	Implemented
AC-009	Image is displayed in student details.	Student Portal renders profile_photo from the current student response.	Implemented
AC-010	Student without image has a placeholder.	Initials are displayed when no photo can be shown.	Implemented
AC-011	Existing Edit and Delete continue to work.	Edit path preserves the current image when unchanged; Delete route remains.	Implemented
AC-012	Admin authentication and authorization remain.	Create/update upload routes are protected by authenticate and ADMIN.	Implemented
AC-013	Unsafe or unsupported formats are blocked.	Extension, MIME, and size filters run on both client and server.	Implemented

Status above indicates that the corresponding implementation is present in the reviewed source. It is not a record of functional test execution.

5. Evidence

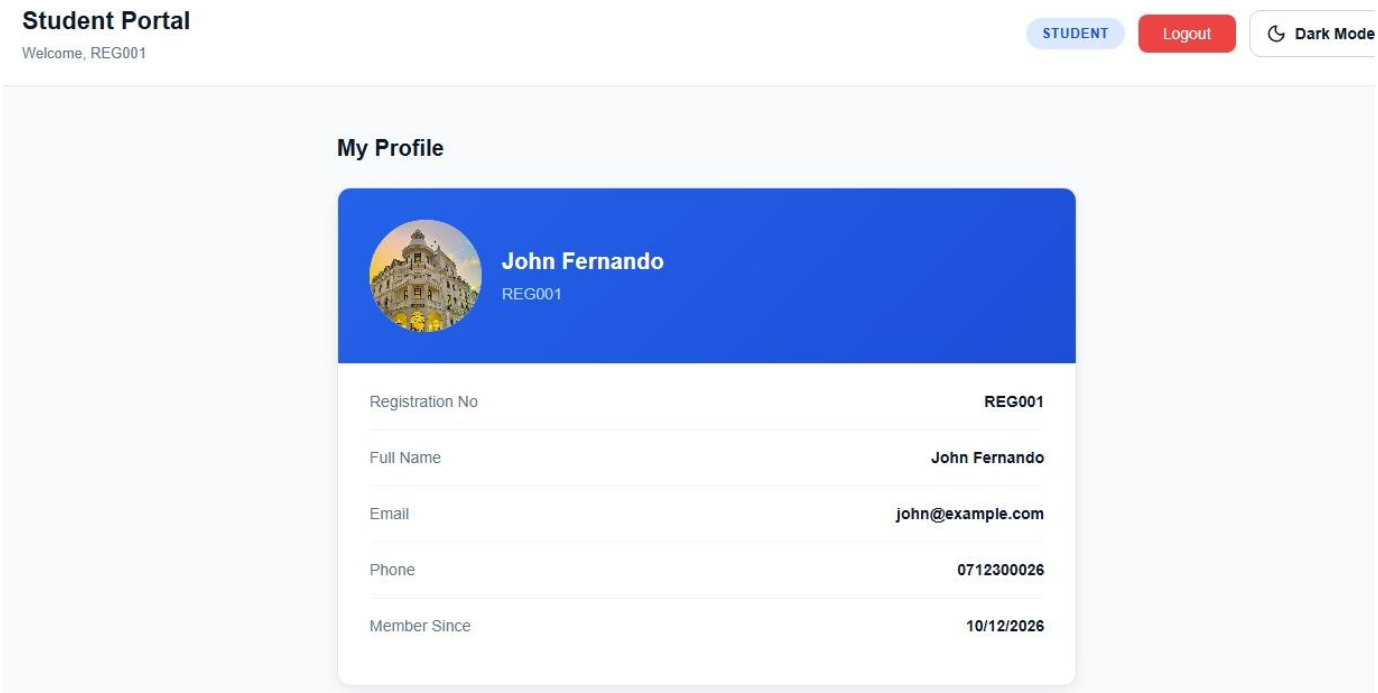
Evidence-01: Student Profile Image Upload

This screenshot demonstrates that the administrator can select a valid PNG, JPG, or JPEG profile image, preview the selected image, and submit the student details with the image.



Evidence-02: Student Profile Image Display

This screenshot demonstrates that the uploaded profile image is displayed correctly in the Student Portal and that the initials fallback is displayed when a student does not have a profile image.



6. Final Result

Item	Result
CR-010 implementation status	Completed
Frontend upload and preview	Implemented
Backend validation and storage	Implemented
Student profile image display	Implemented
Manual test evidence	Completed
Final status	Completed

6.1 Final Summary

CR-010 adds optional student profile image upload to the administrator Add/Edit Student workflow. The application checks supported formats and file size, stores generated image files under backend/Uploads/student, records the relative path against the student profile, and displays the image in the Student Portal. Existing administrator authorization is retained, and edits without a new image preserve the current path. The implementation is marked completed by the requester.

6.2 Issues and Limitations

- The checked-in database/schema.sql does not declare the profile_photo column required by the current model/controller. The connected development database has the column; add it to the schema or a migration before creating a fresh database.

Student Name: N. Bavanika

Student ID: JFICT24_19

Date: 12 October 2026

Signature:

~ END OF REPORT ~